PROYECTO FINAL DE MINERÍA DE DATOS – METODOLOGÍA CRISP-DM

Predicción del precio por noche de alojamientos de Airbnb en Nueva York (2019)

Integrantes: Ismael López Cardozo – Tatiana Fernanda Suarez – Santiago Concha Morales

Repositorio: AIRBNB_streamlit · **Aplicación:** https://airbnbapp-c8wca8smbq7ohcvhcgitgh.streamlit.app/

---

1. ENTENDIMIENTO DEL NEGOCIO

1.1 Descripción del negocio

Airbnb es una plataforma digital de alquiler de alojamientos de corta estancia que conecta a anfitriones, que publican casas, apartamentos o habitaciones, con huéspedes que los reservan por noches. Airbnb cobra una comisión por cada reserva, y el anfitrión fija libremente el precio por noche de su anuncio.

Nueva York es uno de los mercados más grandes de la plataforma: en 2019 contaba con cerca de 49 000 anuncios activos, distribuidos en los cinco distritos de la ciudad (Manhattan, Brooklyn, Queens, Bronx y Staten Island). La mayoría se concentran en Manhattan (44 %) y Brooklyn (41 %), y se reparten casi por igual entre viviendas completas (52 %) y habitaciones privadas (46 %).

1.2 Descripción del problema

Los precios del mercado son muy dispersos: van de 10 a 10 000 USD por noche, con una mediana de 106 USD. Para un anfitrión, en especial uno nuevo, no es fácil decidir cuánto cobrar: un precio demasiado alto reduce la ocupación y uno demasiado bajo deja de percibir ingresos. Tampoco es evidente cuánto influyen en el precio la ubicación, el tipo de alojamiento o las condiciones de reserva.

Problema: no existe una forma objetiva de estimar un precio de referencia para un alojamiento a partir de sus características. **Pregunta de minería:** ¿es posible predecir el precio por noche de un anuncio de Airbnb en Nueva York a partir de su ubicación, tipo de alojamiento, título del anuncio, condiciones de reserva y actividad del anfitrión?

1.3 Objetivos de la minería

- Desarrollar un modelo predictivo de regresión para estimar el precio por noche de un alojamiento de Airbnb en Nueva York, mediante métodos de aprendizaje supervisado y de ensamble, a partir de su ubicación, tipo de alojamiento, título del anuncio, condiciones de reserva y actividad del anfitrión.
- Desplegar el mejor modelo en una aplicación web con interfaz gráfica (Streamlit) que permita a un anfitrión obtener el precio estimado de su alojamiento.

1.4 Diseño de solución

Descripción del problema	Tipo de análisis	Tipo de aprendizaje	Tarea analítica	Requerimiento en los datos	Métodos	Evaluación
Estimar el precio por noche de un alojamiento de Airbnb en Nueva York	Predictivo	Supervisado	Regresión	Datos históricos etiquetados con el precio (variable objetivo numérica) y variables de ubicación, barrio, título, tipo de alojamiento, reservas y anfitrión	Ridge, KNN, árbol de decisión, red neuronal MLP. Ensambles: Random Forest, Gradient Boosting, XGBoost. Validación: división 70/30 y validación cruzada de 10 pliegues. Comparación: ANOVA y Tukey. Ajuste: GridSearch y optimización bayesiana	Medidas de error a minimizar: MAE (criterio principal), MedAE, RMSE, MSE, MAPE y error máximo; complementarias: R^2 log y R^2 en USD

1.5 Valor esperado de la evaluación (línea base)

Como línea base se usa un modelo ingenuo que predice siempre la mediana del precio del conjunto de entrenamiento (107 USD). Evaluado sobre el 30 % de prueba obtiene: MAE = 85.2 USD, MedAE = 47.0 USD, MAPE = 57.8 % y R^2 (escala logarítmica) ≈ 0.

Valor esperado: que el modelo final reduzca el MAE al menos un 25 % respecto a la línea base (MAE ≤ 64 USD) y explique al menos el 50 % de la variabilidad del logaritmo del precio (R^2 log ≥ 0.50). Estos criterios se cumplieron: MAE = 55.8 USD y R^2 log = 0.635 (ver sección 4.3).

1.6 Recursos HW y SW (creación de modelos y despliegue)

Creación de modelos

- **Hardware:** Google Colab, entorno gratuito con CPU (2 núcleos virtuales) y unos 12 GB de RAM; no se requiere GPU. El notebook de modelado se ejecuta en unos 70 minutos, la mayor parte en la hiperparametrización.
- **Software:** Python 3; pandas y NumPy (manipulación de datos); Matplotlib y seaborn (gráficas); ydata-profiling (perfilado); scikit-learn 1.6.1 (modelos, pipelines, validación cruzada, Target Encoding, GridSearchCV); XGBoost 3.4.1; Optuna (optimización bayesiana); SciPy y statsmodels (ANOVA y Tukey); joblib (almacenamiento del modelo).

Despliegue

- **Software:** Streamlit (interfaz gráfica), folium y streamlit-folium (mapa interactivo para seleccionar la ubicación), GitHub (repositorio con los archivos de la aplicación) y Streamlit Community Cloud (publicación de la aplicación en un enlace web). En `requirements.txt` las versiones de scikit-learn y XGBoost se fijan iguales a las del entrenamiento. La aplicación se prueba de forma automática con `streamlit.testing`.
- **Hardware:** servidores de Streamlit Community Cloud (gratuito); no se requiere infraestructura propia. El modelo ocupa unos 3 MB.

2. ENTENDIMIENTO DE LOS DATOS ESPECÍFICOS (TI)

2.1 Ciclo de los datos: generación, almacenamiento, modificación, periodicidad

- **Generación:** los datos se generan en la plataforma Airbnb: los anfitriones crean y editan sus anuncios (ubicación, tipo, precio, noches mínimas, disponibilidad) y los huéspedes dejan reseñas tras su estadía. El proyecto Inside Airbnb recopila esta información pública del sitio web mediante extracción automática (web scraping).
- **Almacenamiento:** el conjunto se distribuye como un único archivo CSV, `AB_NYC_2019.csv` (48 895 registros, 16 variables, unos 7 MB), publicado en Kaggle a partir de Inside Airbnb. En el proyecto se almacena en el repositorio y en el entorno de Google Colab.
- **Modificación:** en la plataforma, los anfitriones pueden cambiar el precio y la disponibilidad en cualquier momento, y las reseñas se acumulan con el tiempo. El archivo es una fotografía estática del mercado a julio de 2019 (la reseña más reciente es del 8 de julio de 2019).
- **Periodicidad:** Inside Airbnb publica nuevas extracciones de forma periódica. Para este proyecto se usa una única extracción (2019); para mantener el modelo vigente habría que reentrenarlo con extracciones recientes (ver sección 5).

2.2 Diccionario de datos

Tabla 1. Diccionario de datos del archivo AB_NYC_2019.csv

Variable	Tipo	Descripción
id	Numérica entera (código)	Identificador único del anuncio
name	Texto	Nombre o título del anuncio
host_id	Numérica entera (código)	Identificador del anfitrión
host_name	Texto	Nombre del anfitrión
neighbourhood_group	Categórica (5)	Distrito: Manhattan, Brooklyn, Queens, Bronx, Staten Island
neighbourhood	Categórica (221)	Barrio en el que se ubica el alojamiento
latitude	Numérica continua	Latitud del alojamiento (grados)
longitude	Numérica continua	Longitud del alojamiento (grados)
room_type	Categórica (3)	Tipo de alojamiento: Entire home/apt, Private room, Shared room
price	Numérica entera (USD)	Precio por noche. Variable objetivo
minimum_nights	Numérica entera	Número mínimo de noches por reserva
number_of_reviews	Numérica entera	Número total de reseñas recibidas
last_review	Fecha	Fecha de la reseña más reciente
reviews_per_month	Numérica continua	Promedio de reseñas por mes
calculated_host_listings_count	Numérica entera	Número de anuncios que tiene el anfitrión en la plataforma
availability_365	Numérica entera	Días del año en que el alojamiento está disponible para reservar (0–365)

2.3 Reglas de calidad

Tabla 2. Reglas de calidad definidas desde el negocio

Variable	Regla de calidad (negocio)
id	Único y no nulo
neighbourhood_group	Manhattan, Brooklyn, Queens, Bronx, Staten Island
room_type	Entire home/apt, Private room, Shared room
latitude	40.49 – 40.92 (límites de la ciudad de Nueva York)
longitude	−74.26 – −73.70 (límites de la ciudad de Nueva York)
price	Mayor que 0 USD
minimum_nights	1 – 365 noches
number_of_reviews	Entero mayor o igual a 0
reviews_per_month	Mayor o igual a 0; igual a 0 si number_of_reviews = 0
calculated_host_listings_count	Entero mayor o igual a 1
availability_365	0 – 365 días
last_review	Fecha anterior o igual a la fecha de extracción (julio de 2019)

3. PREPARACIÓN DE DATOS

3.1 Integración

Los datos provienen de una única fuente, el archivo `AB_NYC_2019.csv`, que se cargó con pandas: 48 895 registros y 16 variables. No fue necesario integrar varias fuentes. Se verificó que no hay identificadores (id) repetidos y las variables de texto se convirtieron al tipo categórico.

3.2 Selección de variables

- **Variables irrelevantes eliminadas:** id y host_id (códigos) y host_name (nombre de la persona anfitriona).
- **Variables conservadas para la ingeniería de características:** name y neighbourhood. name no es el nombre de una persona, sino el título del anuncio, y sus palabras (luxury, penthouse, cozy) indican el segmento del alojamiento. neighbourhood no es redundante con las coordenadas, porque el precio cambia entre barrios vecinos; su alta cardinalidad (221 categorías) impide crear dummies, por lo que se codifica con Target Encoding. Al ser texto y una categórica de alta cardinalidad, quedan fuera de los análisis de correlación de la preparación.

Quedan 13 variables: name, neighbourhood_group, neighbourhood, latitude, longitude, room_type, price, minimum_nights, number_of_reviews, last_review, reviews_per_month, calculated_host_listings_count y availability_365.

Uso en la ingeniería de características. `name` y `neighbourhood` se transforman en la etapa de ingeniería de características. La exploración del notebook `04_mineria_caracteristicas`, que parte del conjunto preparado, mostró que aportan información: el barrio codificado tiene una correlación de 0.52 con el logaritmo del precio, y los anuncios con palabras de lujo en el título tienen una mediana de 175 USD frente a 100 USD del resto. Las nuevas variables se calculan dentro del pipeline del modelo (sección 3.8). El problema de alta cardinalidad del barrio se resuelve con Target Encoding, que lo convierte en una sola columna.

3.3 Descripción estadística

Tabla 3. Descripción estadística de las variables numéricas (48 895 registros)

Variable	Media	Mediana	Desv. estándar	Mínimo	Máximo
price (USD)	152.72	106.00	240.15	0	10 000
minimum_nights	7.03	3.00	20.51	1	1 250
number_of_reviews	23.27	5.00	44.55	0	629
reviews_per_month	1.37	0.72	1.68	0.01	58.50
calculated_host_listings_count	7.14	1.00	32.95	1	327
availability_365	112.78	45.00	131.62	0	365
latitude	40.73	40.72	0.05	40.50	40.91
longitude	−73.95	−73.96	0.05	−74.24	−73.71

- **price** es muy asimétrica: la media (152.7 USD) es mucho mayor que la mediana (106 USD) y el máximo llega a 10 000 USD. También son asimétricas minimum_nights, number_of_reviews y calculated_host_listings_count: la mayoría de los valores son bajos, con pocos valores muy altos.
- **availability_365:** la mitad de los anuncios están disponibles 45 días o menos al año, y muchos tienen 0 días disponibles.
- **Categóricas:** Manhattan (44.3 %) y Brooklyn (41.1 %) concentran la mayoría de los anuncios, seguidos de Queens (11.6 %), Bronx (2.2 %) y Staten Island (0.8 %). Por tipo, predominan las viviendas completas (52.0 %) y las habitaciones privadas (45.7 %); las compartidas son el 2.4 %.
- **last_review:** las reseñas más recientes van de 2011 a julio de 2019, con la mayoría en 2019.

El perfilado completo de los datos (ydata-profiling) se guardó en `docs/output.html`.

3.4 Limpieza de atípicos – errores

Se comparan los valores observados con las reglas de calidad de la Tabla 2. Solo se eliminan los valores que son errores. Los valores atípicos legítimos (por ejemplo, alojamientos de lujo con precios de miles de dólares) se conservan, porque representan fenómenos reales del mercado y eliminarlos haría que el modelo ignorara ese segmento.

Variable	Regla de calidad	Mínimo – máximo / categorías de los datos	Indicativo de error
price	> 0 USD	0 – 10 000	Error: 11 registros con precio 0 (eliminados). Precios altos: reales, se conservan
minimum_nights	1 – 365	1 – 1 250	Error: 14 registros con más de 365 noches (eliminados)
latitude	40.49 – 40.92	40.50 – 40.91	Sin error
longitude	−74.26 – −73.70	−74.24 – −73.71	Sin error
number_of_reviews	≥ 0	0 – 629	Sin error
reviews_per_month	≥ 0	0.01 – 58.50	Sin error
calculated_host_listings_count	≥ 1	1 – 327	Sin error

availability_365	0 – 365	0 – 365	Sin error
neighbourhood_group	5 distritos	5 distritos	Sin error
room_type	3 tipos	3 tipos	Sin error

Se eliminaron 25 registros con errores; el conjunto queda con 48 870 registros.

3.5 Limpieza de nulos

La estrategia en la limpieza de nulos es la siguiente:

- Eliminar los registros con más del 15 % de valores faltantes.
- Eliminar las variables con más del 15 % de valores faltantes.
- Imputar variables con máximo 15 % de valores faltantes: media (numérica simétrica), mediana (numérica asimétrica), moda (categórica).

Variable	Limpieza de nulos
reviews_per_month	10 043 nulos (20.5 %). Imputada con 0: son nulos estructurales, que coinciden exactamente con los anuncios sin reseñas (number_of_reviews = 0). El valor real se conoce con certeza (0 reseñas por mes), por lo que no aplica el límite del 15 %.
last_review	10 043 nulos (20.5 %). Eliminada por superar el 15 % de nulos; además, al ser una fecha, no existe un valor de imputación con sentido para un anuncio sin reseñas.
name, host_name	16 y 21 nulos en el archivo original. host_name ya se había eliminado. Los títulos vacíos (menos del 0.1 %) se imputan con texto vacío, que equivale a un título sin palabras clave.
Demás variables	Sin valores nulos.

Después de la limpieza, el conjunto no tiene valores nulos: 48 870 registros y 12 variables.

3.6 Análisis de correlaciones para redundancia

Se calculó la matriz de correlación de Pearson entre todas las variables (las categóricas, convertidas en variables dummy). Se considera redundante un par de variables distintas con correlación absoluta mayor que 0.8.

Tabla 4. Correlaciones más altas entre variables

Par de variables	|r|	Interpretación
room_type_Private room – room_type_Entire home/apt	0.95	Dummies de la misma variable; son complementarias, no redundantes
neighbourhood_group_Brooklyn – neighbourhood_group_Manhattan	0.75	Dummies de la misma variable
latitude – neighbourhood_group_Brooklyn	0.67	Relación esperada (el distrito depende de la ubicación); < 0.8
longitude – neighbourhood_group_Queens	0.62	Relación esperada; < 0.8
number_of_reviews – reviews_per_month	0.59	Relación moderada; < 0.8

Ningún par de variables distintas supera el umbral, por lo que no se elimina ninguna variable por redundancia.

3.7 Análisis de correlaciones para irrelevancia (predicciones)

Se buscan variables con correlación muy baja con el precio ($|r| < 0.05$). Como la correlación de Pearson solo mide relaciones lineales y es muy sensible a los valores extremos del precio, se verificó también la correlación de

Spearman, basada en rangos. Para las categóricas se calculó el promedio ponderado de la correlación absoluta de sus dummies.

Tabla 5. Correlación de cada variable con el precio

Variable	Pearson	Spearman	Decisión
latitude	0.034	0.136	Se conserva
longitude	−0.150	−0.438	Se conserva
minimum_nights	0.054	0.101	Se conserva
number_of_reviews	−0.048	−0.055	Se conserva
reviews_per_month	−0.051	−0.060	Se conserva
calculated_host_listings_count	0.057	−0.106	Se conserva
availability_365	0.082	0.086	Se conserva
neighbourhood_group (ponderada)	0.123	—	Se conserva
room_type (ponderada)	0.244	—	Se conserva

- **latitude** tiene una correlación de Pearson baja (0.03), pero no se elimina: su correlación de Spearman es mayor (0.14) y su efecto sobre el precio no es lineal, sino que depende de su combinación con la longitud (cercanía al centro de Manhattan). Ambas coordenadas se conservan y se transforman en la ingeniería de características.
- **number_of_reviews** tiene Pearson en el límite (−0.048), pero con Spearman supera el umbral (−0.055), al igual que las demás variables.

En consecuencia, no se elimina ninguna variable por irrelevancia. El conjunto preparado (`Airbnb_NYC_2019_limpio.csv`) tiene 48 870 registros y 12 variables, y es la entrada de la minería de características y del entrenamiento de todos los modelos.

3.8 Ingeniería de características

3.8.1 Balanceo (clasificación)

El balanceo se aplica en problemas de clasificación con clases desiguales. Este es un problema de regresión (price es numérica continua), por lo que no hay clases que balancear y el balanceo no aplica. El problema equivalente, la asimetría del precio, se trata con la transformación logarítmica: en el 70 % de entrenamiento la asimetría pasa de 18.1 a 0.61.

3.8.2 Creación de nuevas variables

A partir de las coordenadas, el título y la disponibilidad se crean 18 variables. Las funciones están en `features.py` y se ejecutan dentro del pipeline.

Tabla 6. Variables creadas en la ingeniería de características

Familia	Variables creadas	Justificación	Spearman con el precio
Distancias haversine a 6 puntos de interés y al aeropuerto más cercano	dist_times_square_km, dist_central_park_km, dist_fidi_km, dist_brooklyn_bridge_km, dist_jfk_km, dist_lga_km, dist_min_aeropuerto_km	Las coordenadas por separado tienen una relación débil y no lineal con el precio; las distancias a lugares de referencia la resumen	Times Square −0.46; Financial District −0.36

Señales del título del anuncio	feat_luxury_count, feat_has_luxury, feat_budget_count, feat_has_budget, feat_transit_count, feat_spacious_count, name_length, name_word_count	Palabras como luxury, penthouse, cozy o small indican el segmento del alojamiento	Lujo 0.22; economía −0.14
Ratios de negocio	availability_ratio, is_full_year_available, is_host_multi_listing	Disponibilidad anual y anfitrión profesional	Multi-anuncio −0.13

Se conservan también la latitud y la longitud, que los modelos de árboles aprovechan para identificar zonas.

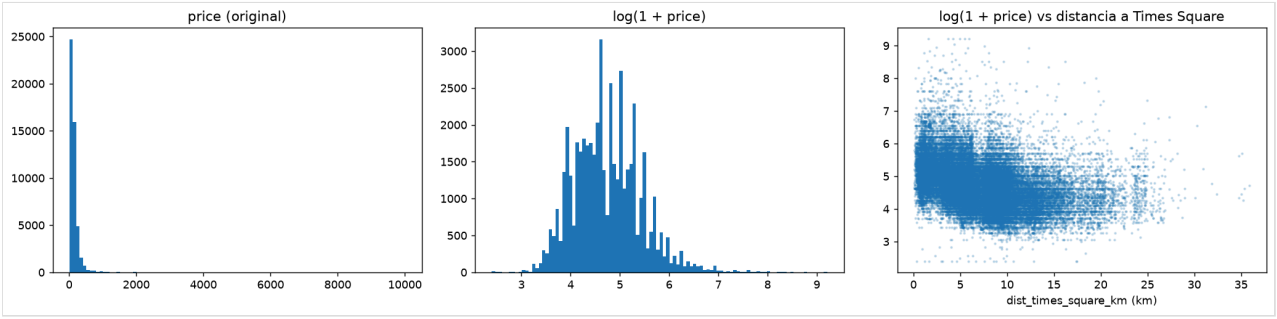


Figura 1. Distribución del precio original, de log(1 + price) y relación con la distancia a Times Square

3.8.3 Reducción de dimensión (opcional en predicciones)

No se aplicó. Después de la codificación el modelo trabaja con 34 características para 34 209 registros de entrenamiento, por lo que no hay problema de alta dimensionalidad, y conservar las variables originales mantiene la interpretabilidad.

3.8.4 Transformaciones

- **Logaritmo del precio (variable objetivo):** los modelos se entrenan con log(1 + price) y la predicción se devuelve a dólares con la transformación inversa; los precios extremos dejan de dominar el entrenamiento.
- **Logaritmo de los conteos:** log(1 + x) en minimum_nights, number_of_reviews, reviews_per_month y calculated_host_listings_count, por ser muy asimétricas.
- **Target Encoding de neighbourhood:** cada barrio se reemplaza por la media suavizada del logaritmo del precio, calculada con validación cruzada interna de 5 pliegues para evitar fuga de información. Así se usan los 221 barrios sin crear 221 columnas.
- **Codificación one-hot** de neighbourhood_group y room_type.
- **Estandarización** (media 0, desviación 1) de las 25 variables numéricas, necesaria para KNN, la red neuronal y la regresión Ridge.

Todas las transformaciones están dentro de un pipeline de scikit-learn, de modo que en cada pliegue de la validación cruzada se ajustan solo con los datos de entrenamiento y no hay fuga de información.

4. MODELAMIENTO, EVALUACIÓN E INTERPRETACIÓN

4.1 Configuración de métodos de machine learning

El modelado parte del conjunto preparado `Airbnb_NYC_2019_limpio.csv`, sin volver a aplicar las reglas de calidad (solo se comprueba que se cumplen). Los datos se dividieron en 70 % de entrenamiento (34 209 registros) y 30 % de prueba (14 661 registros), con semilla 42. Con el 70 % se aplicó validación cruzada de 10 pliegues (K-Fold con mezcla aleatoria). La validación cruzada no es una medida de evaluación, sino la técnica de validación: divide el 70 % en 10 partes, entrena cada modelo 10 veces con 9 partes y lo evalúa con la parte restante, lo que da 10 valores de cada medida por modelo para aplicar ANOVA y Tukey. Se usaron 4 métodos de aprendizaje supervisado y 3 métodos de ensamble, todos dentro del mismo pipeline de 34 características:

Tabla 7. Configuración inicial de los métodos

Método de IA	Configuración de parámetros
Regresión Ridge	alpha = 1.0 (regularización L2)
K vecinos más cercanos (KNN)	n_neighbors = 15, pesos uniformes, distancia euclidiana
Árbol de decisión	max_depth = 10, min_samples_leaf = 20
Red neuronal (MLP)	2 capas ocultas (64 y 32 neuronas), activación ReLU, optimizador Adam, early_stopping, max_iter = 300
Random Forest (ensamble – bagging)	100 árboles, min_samples_leaf = 5
Gradient Boosting (ensamble – boosting)	100 árboles, learning_rate = 0.1, max_depth = 3
XGBoost (ensamble – boosting)	300 árboles, learning_rate = 0.05, max_depth = 6

4.2 Análisis de medidas de calidad

Como es un problema de regresión, el objetivo es **minimizar el error**. Para cada modelo se calcularon 8 medidas en los 10 pliegues: 6 de error (menor es mejor) y 2 de R^2 (mayor es mejor). La tabla muestra el promedio, ordenado de menor a mayor MAE; el tiempo es el total de los 10 pliegues.

Tabla 8. Medidas de calidad en la validación cruzada de 10 pliegues (promedio)

Método	MAE (USD)	MedAE (USD)	RMSE (USD)	MSE (USD²)	MAPE	Error máx. (USD)	R^2 log	R^2 USD	Tiempo (s)
XGBoost	**55.06**	**23.41**	**203.40**	**43 778**	**0.313**	**6 305**	**0.620**	**0.193**	**9.6**
Random Forest	55.31	23.58	203.90	43 999	0.316	6 319	0.612	0.189	192.1
Gradient Boosting	56.63	24.33	207.41	45 412	0.321	6 388	0.599	0.159	135.4
Red neuronal (MLP)	57.63	25.11	205.77	44 691	0.333	6 341	0.587	0.172	22.4
Regresión Ridge	58.59	25.57	209.04	46 051	0.335	6 407	0.569	0.145	2.6
Árbol de decisión	58.81	25.47	206.78	45 103	0.343	6 344	0.563	0.164	5.6

KNN	60.70	27.59	209.63	46 299	0.369	6 405	0.513	0.140	3.7

Interpretación de las medidas

- **MAE (55.1 – 60.7 USD):** con XGBoost la predicción se aleja en promedio 55 USD del precio real. Es el criterio principal de selección porque está en dólares y no lo dominan los precios extremos.
- **MedAE (23.4 – 27.6 USD):** para la mitad de los alojamientos, el error de XGBoost es menor que 23.4 USD; describe el error de un alojamiento típico.
- **RMSE (203 – 210 USD) y MSE (43 778 – 46 299 USD²):** el RMSE es unas 3.7 veces el MAE; al elevar los errores al cuadrado, los dominan unos pocos alojamientos de miles de dólares. El MSE, en dólares al cuadrado, no se interpreta directamente.
- **MAPE (0.313 – 0.369):** con XGBoost el error equivale en promedio al 31 % del precio.
- **Error máximo (6 305 – 6 407 USD, promedio de los pliegues):** casi igual en los 7 modelos; corresponde a alojamientos de miles de dólares que ningún modelo anticipa con estas variables.
- **R^2 en escala logarítmica (0.51 – 0.62):** XGBoost explica el 62 % de la variabilidad del logaritmo del precio, la escala en la que aprende; faltan variables decisivas como el tamaño o el número de habitaciones.
- **R^2 en dólares (0.14 – 0.20):** mucho más bajo por los mismos precios extremos.
- **Tiempo de aprendizaje:** XGBoost es el más preciso y de los más rápidos (10 s), frente a 192 s de Random Forest y 135 s de Gradient Boosting.

XGBoost es el mejor en las 8 medidas y Random Forest el segundo en casi todas. Con las 34 características, el MAE de XGBoost en validación cruzada baja de 55.9 a 55.1 USD respecto a la versión de 9 variables; KNN empeora (de 58.6 a 60.7 USD) porque con muchas variables las distancias entre alojamientos pierden significado. En los diagramas de caja, la variación entre pliegues es comparable a las diferencias entre modelos, por lo que se necesita una prueba estadística.

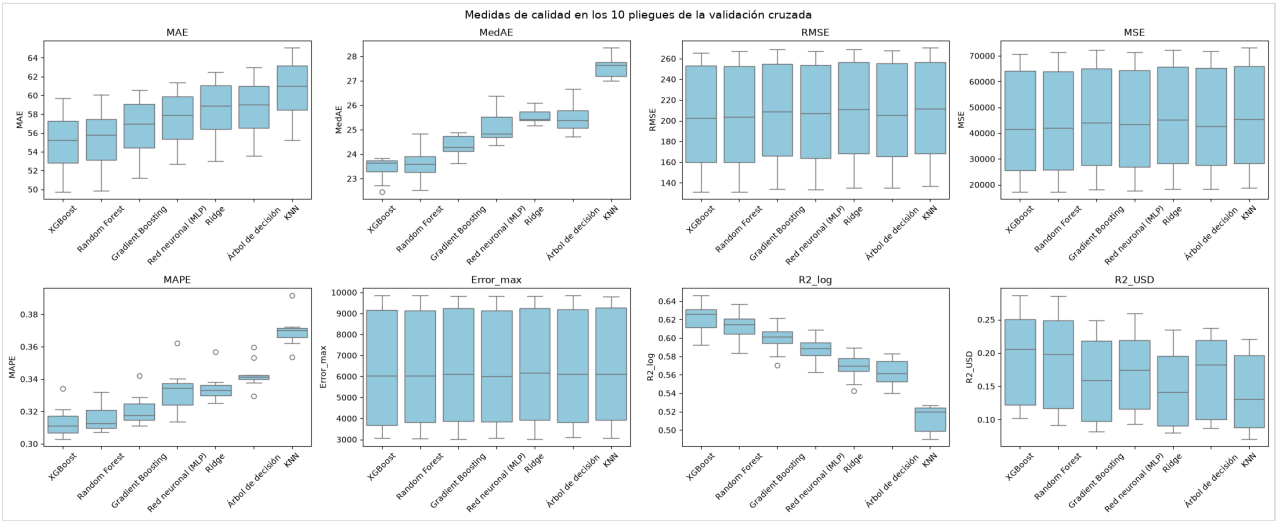


Figura 2. Medidas de calidad en los 10 pliegues de la validación cruzada

4.3 Selección del mejor modelo

Criterio de selección. El mejor modelo es el de menor MAE, en tres etapas: (1) selección de los 3 mejores modelos con ANOVA y Tukey; (2) hiperparametrización de esos 3 modelos con GridSearch y optimización bayesiana, minimizando el MAE; (3) confirmación con el 30 % de prueba.

Etapa 1 – ANOVA y Tukey. Se compararon los MAE de los 10 pliegues de los 7 modelos ($\alpha = 0.05$).

- **ANOVA de un factor:** F = 4.27, p-valor = 0.0011 < 0.05. Se rechaza la hipótesis nula: no todos los modelos tienen el mismo MAE medio.
- **Tukey (HSD):** de las 21 comparaciones por pares, 2 son significativas: XGBoost vs. KNN (diferencia de 5.6 USD, p = 0.003) y Random Forest vs. KNN (5.4 USD, p = 0.004). Las demás diferencias no son significativas.
- **Modelos seleccionados:** XGBoost y Random Forest, con el menor MAE y únicos significativamente mejores que otro modelo (entre ellos p = 1.0); y Gradient Boosting, tercero en MAE (56.7 USD). Su diferencia con la red neuronal (57.7 USD) no es significativa; se eligió porque la supera en MAE, MedAE, MAPE y R^2 log, está cerca de ser significativamente mejor que KNN (p = 0.067) y usaba su configuración por defecto, por lo que podía mejorar más con la hiperparametrización.

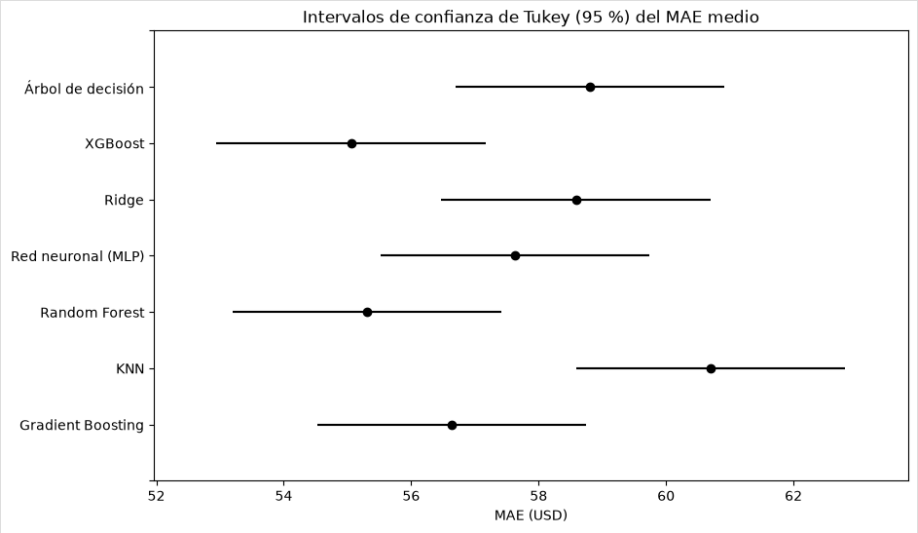


Figura 3. Intervalos de confianza de Tukey (95 %) del MAE medio de cada modelo

Etapa 2 – Hiperparametrización. Cada modelo se optimizó con GridSearchCV (rejilla de 4 a 8 combinaciones) y con optimización bayesiana (Optuna, algoritmo TPE: 20 intentos en XGBoost, 15 en Random Forest y 12 en Gradient Boosting), ambas con validación cruzada de 3 pliegues sobre el 70 % y minimizando el MAE. **Etapa 3:** los 6 modelos resultantes se evaluaron con el 30 % de prueba.

Tabla 9. Resultados de la hiperparametrización: MAE estimado con validación cruzada y medidas sobre el 30 % de prueba (errores en USD; MSE en USD²)

Modelo	Técnica	MAE (valid. cruzada)	MAE	MedAE	RMSE	MSE	MAPE	Error máx.	R^2 log
XGBoost	**Bayesiana (Optuna)**	**54.89**	**55.83**	**22.71**	**245.02**	**60 036**	**0.304**	**9 939**	**0.635**
XGBoost	GridSearch	55.08	55.69	22.42	243.36	59 226	0.305	9 954	0.635
Random Forest	Bayesiana (Optuna)	55.41	56.42	23.20	246.19	60 610	0.309	9 948	0.623
Random Forest	GridSearch	55.51	56.42	23.09	246.60	60 813	0.308	9 946	0.623
Gradient Boosting	Bayesiana (Optuna)	55.59	56.29	23.01	244.84	59 947	0.308	9 954	0.628
Gradient Boosting	GridSearch	56.06	56.98	23.37	246.96	60 990	0.311	9 943	0.621

- La optimización bayesiana superó a GridSearch en validación cruzada en los 3 modelos, porque explora valores continuos y concentra los intentos en las zonas más prometedoras.

- Gradient Boosting fue el que más mejoró (MAE de 56.7 a 55.6 USD); XGBoost y Random Forest mejoraron menos porque su configuración inicial ya era buena.
- En la prueba, XGBoost con GridSearch tiene un MAE apenas menor (55.69 frente a 55.83 USD). La diferencia (0.14 USD) es mucho menor que la variación entre pliegues, y la regla de selección se fijó antes con la validación cruzada: elegir con el conjunto de prueba haría que su error dejara de ser una estimación imparcial. Por eso se conserva el modelo de Optuna.
- El MAE de prueba (55.8 USD) es casi igual al estimado con validación cruzada (54.9 USD), y el R^2 log también (0.635 frente a 0.620): no hay sobreajuste.

Mejor modelo: XGBoost optimizado con optimización bayesiana (Optuna).

Tabla 10. Configuración final del modelo

Método de IA	Configuración de parámetros final
XGBoost (XGBRegressor), dentro del pipeline de 34 características y con objetivo log(1 + price)	n_estimators = 355; learning_rate = 0.0298; max_depth = 8; subsample = 0.95; colsample_bytree = 0.51; min_child_weight = 20

Usar la mitad de las variables en cada árbol (colsample_bytree = 0.51) y exigir un peso mínimo de 20 por hoja (min_child_weight = 20) regulariza el modelo y reduce el sobreajuste.

Evaluación final con el 30 % de prueba. Se compara el modelo final con la línea base ingenua (predecir siempre la mediana, 107 USD) y con el modelo de la primera versión del proyecto (XGBoost optimizado sobre 9 variables, entrenado con el mismo 70 %):

Tabla 11. Comparación en el 30 % de prueba

Medida	Línea base ingenua	Modelo base (9 variables)	Modelo final (34 características)	Mejora frente al modelo base
MAE (USD)	85.16	56.92	**55.83**	−1.9 %
MedAE (USD)	47.00	22.97	**22.71**	−1.1 %
RMSE (USD)	268.03	245.45	**245.02**	−0.2 %
MSE (USD^2)	71 842	60 245	**60 036**	−0.3 %
MAPE	0.578	0.310	**0.304**	−2.2 %
Error máximo (USD)	9 893	9 949	**9 939**	−0.1 %
R^2 log	—	0.621	**0.635**	+2.4 %
R^2 USD	−0.031	0.135	**0.138**	+2.2 %

- Frente a la línea base ingenua, el modelo reduce el MAE un 34 % y el error mediano a la mitad, y cumple el valor esperado de la sección 1.5 (MAE ≤ 64 USD y R^2 log ≥ 0.50).
- Frente al modelo de 9 variables, la minería de características mejora todas las medidas de error. La mejora es modesta (1.9 % en MAE, 1.1 % en MedAE y 2.2 % en MAPE) y está sobre todo en el error promedio, no en el de un alojamiento típico.
- El R^2 en dólares (0.138), el MSE y el RMSE (245 USD) son peores que los estimados con validación cruzada porque la prueba contiene alojamientos de 5 000 a 10 000 USD, que dominan las medidas basadas en errores al cuadrado. Las medidas robustas (MAE, MedAE, MAPE y R^2 log) se mantienen estables.

- **Error máximo (9 939 USD):** corresponde al alojamiento de 10 000 USD por noche del conjunto de prueba, al que el modelo asigna un precio normal. Es un caso aislado: el 75 % de las predicciones tiene un error menor de 50 USD y el 96.4 %, menor de 200 USD.
- **En la práctica:** para un alojamiento típico el modelo se equivoca unos 23 USD, con un error relativo medio del 30 %. Sirve como precio de referencia para alojamientos comunes, pero no para el segmento de lujo, cuyos precios dependen de variables que el conjunto no incluye.
- **Real vs. predicho:** los puntos siguen la diagonal, pero el modelo subestima los alojamientos muy caros y sobreestima los muy baratos, lo esperable cuando faltan variables que expliquen los extremos.
- **Variables más importantes:** el tipo de alojamiento (vivienda completa 0.55, habitación privada 0.20, compartida 0.05), estar en Manhattan (0.06) y el micro-barrio codificado (0.016). Entre las variables creadas, las más útiles son las palabras de lujo del título (0.010), la disponibilidad todo el año (0.008) y la distancia a Times Square (0.008).

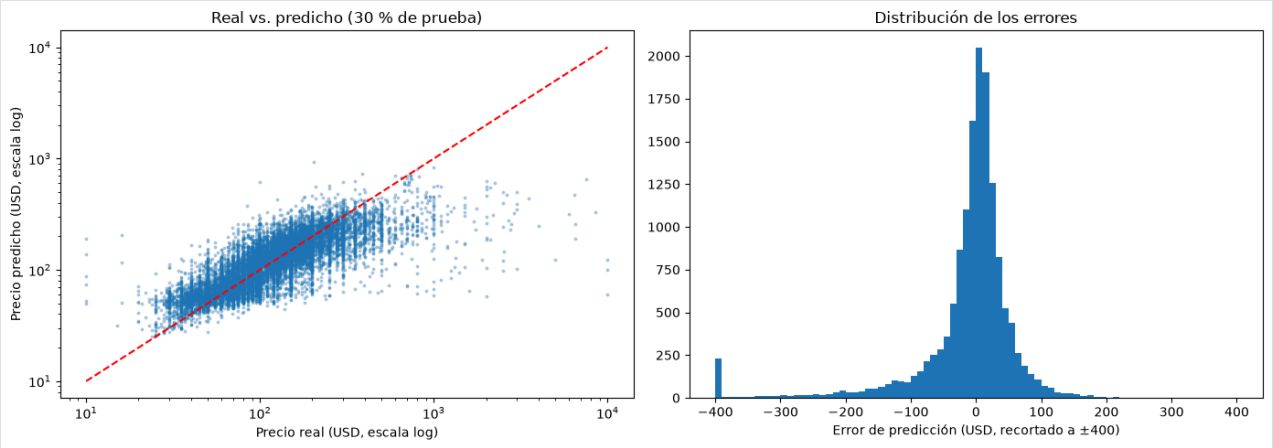


Figura 4. Precio real vs. predicho y distribución de los errores en el 30 % de prueba

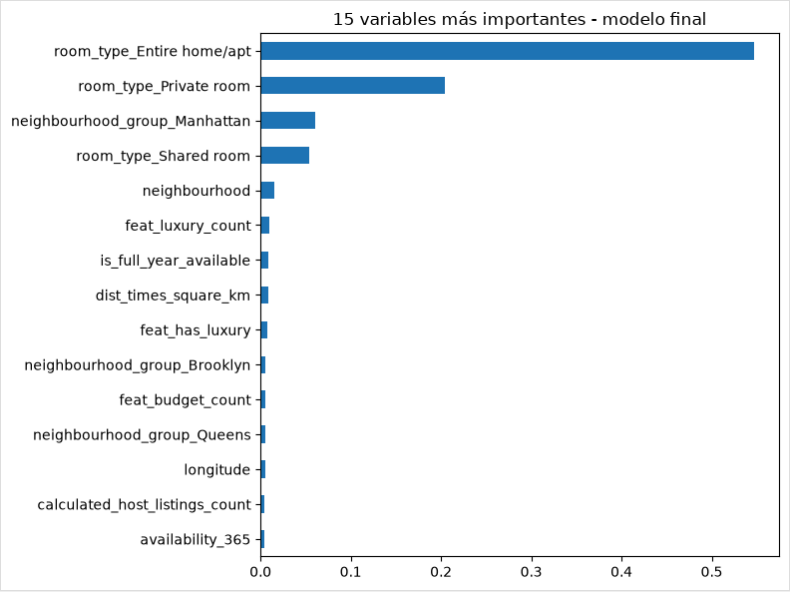


Figura 5. Las 15 variables más importantes del modelo final (XGBoost)

5. DESPLIEGUE

5.1 Predicción de datos futuros: interfaz para usar el modelo

El modelo final se almacenó con joblib en un pipeline (`models/modelo_final_airbnb.pkl`, unos 3 MB) que incluye todas las operaciones de preparación de los datos:

Tabla 12. Pasos del pipeline final

Paso	Clase	Operación
1. Preparación de datos	FunctionTransformer (preparar_datos)	Conserva las 11 columnas de entrada (descarta id, host_id, host_name y last_review), rellena reviews_per_month y name vacíos y aplica a los registros nuevos las mismas reglas de calidad de la preparación (minimum_nights entre 1 y 365, availability_365 entre 0 y 365)
2. Ingeniería de características	FunctionTransformer (crear_variables)	Distancias a 6 puntos de interés, señales del título, ratios de negocio y log(1 + x) de los conteos
3. Preprocesamiento	ColumnTransformer	Estandarización, one-hot de neighbourhood_group y room_type, y Target Encoding de neighbourhood
4. Modelo	XGBRegressor	Modelo final optimizado con Optuna
5. Transformación del objetivo	TransformedTargetRegressor	Entrena con log(1 + price) y devuelve la predicción en dólares

Las funciones de los pasos 1 y 2 están en `features.py`, necesario para cargar el modelo. En el notebook de modelado se comprobó que el pipeline guardado da exactamente las mismas predicciones que el modelo en memoria en el 30 % de prueba, y que prepara por sí mismo registros con el formato del archivo original (con identificadores, reviews_per_month vacío y 1 000 noches mínimas) antes de predecir.

La interfaz se construyó con Streamlit (notebook `Despliegue_Airbnb_NYC`) y se publica en Streamlit Community Cloud desde el repositorio de GitHub, que contiene `app.py`, `features.py`, `compat.py` (carga del modelo en Windows), `models/modelo_final_airbnb.pkl`, `models/metricas_modelo.json`, `data/ubicaciones_distritos.csv`, `data/Airbnb_NYC_2019_limpio.csv`, `data/titulos_ejemplo.csv`, el tema visual `.streamlit/config.toml` y `requirements.txt` con todas las versiones fijadas. La integración continua de GitHub ejecuta las pruebas en Linux y Windows con Python 3.12 y 3.13. La app guía al usuario en tres pasos:

1. **Ubicación.** Marca el alojamiento con un clic en un mapa interactivo (folium y streamlit-folium) que muestra los 6 puntos de interés del modelo. La app asigna el distrito y el micro-barrio del alojamiento del dataset más cercano y muestra las distancias a Times Square, Central Park, Financial District, Brooklyn Bridge y el aeropuerto más cercano, las mismas que calcula el pipeline. Si el mapa no carga, el usuario puede escribir las coordenadas o buscar una dirección (Nominatim, OpenStreetMap), y corregir a mano el distrito y el micro-barrio, que la app estima con el centroide del barrio más cercano.
2. **El anuncio.** Elige el título de una lista de 50 ejemplos o lo escribe; la app resalta en vivo las palabras clave que lee el modelo (lujo, espacio, transporte y economía). Después elige el tipo de alojamiento e ingresa las noches mínimas, la disponibilidad, las reseñas y el número de anuncios del anfitrión.
3. **Estimación.** Obtiene el precio sugerido por noche y un rango de referencia, que corresponde a la mitad central de los errores del modelo en el 30 % de prueba (entre 0.78 y 1.22 veces el precio estimado). El precio

se compara con la mediana de los anuncios del mismo micro-barrio y tipo de alojamiento, tomada del conjunto de datos limpio, y se marca sobre el histograma de precios de esa zona.

¿Qué pasaría si...? En lugar de una explicación con reglas escritas a mano, la app recalcula el precio con el mismo modelo cambiando una sola cosa: otro tipo de alojamiento, el título con o sin palabras de lujo, el mínimo de 30 noches que exige la Local Law 18 y el doble de reseñas. Así el usuario ve cuánto pesa cada decisión en la estimación. En la barra lateral se resumen las métricas del modelo (MAE, MedAE y R^2), las fases de CRISP-DM y el aviso regulatorio.

El notebook de despliegue prueba el modelo con alojamientos de ejemplo y ejecuta la aplicación de forma automática con el probador de Streamlit (AppTest): escribe un título, presiona *Estimar precio* y comprueba que la app responde sin errores.

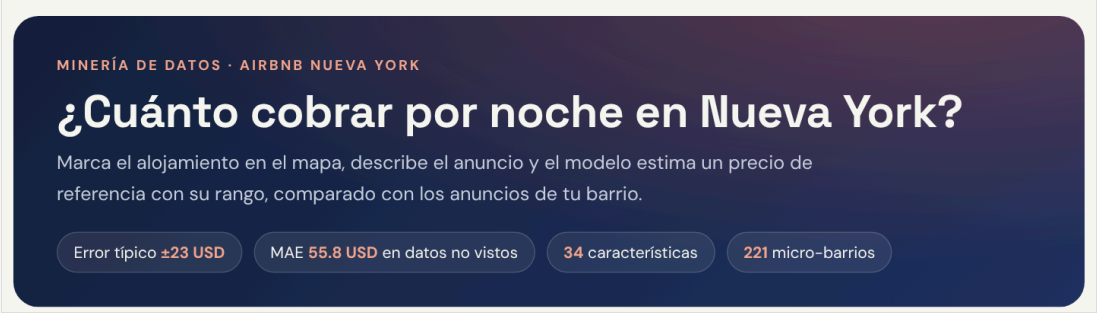


Figura 6a. Encabezado de la app con las métricas del modelo

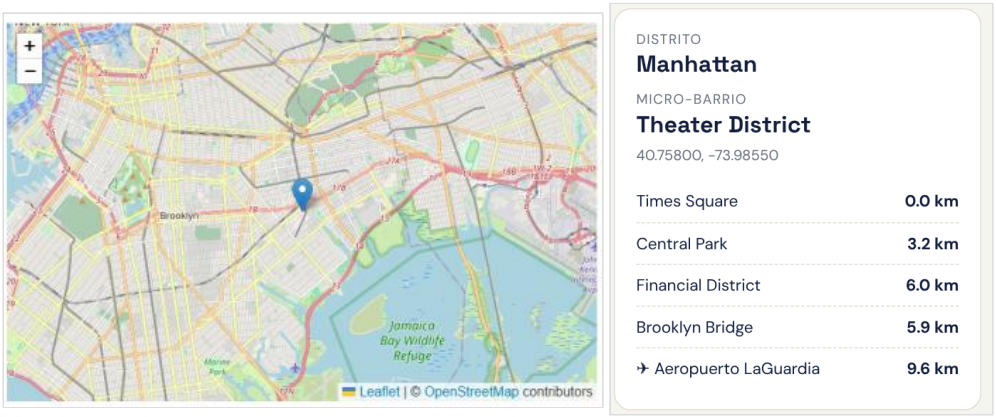


Figura 6b. Paso 1: clic en el mapa interactivo; la app muestra el distrito, el micro-barrio y las distancias a los puntos de interés

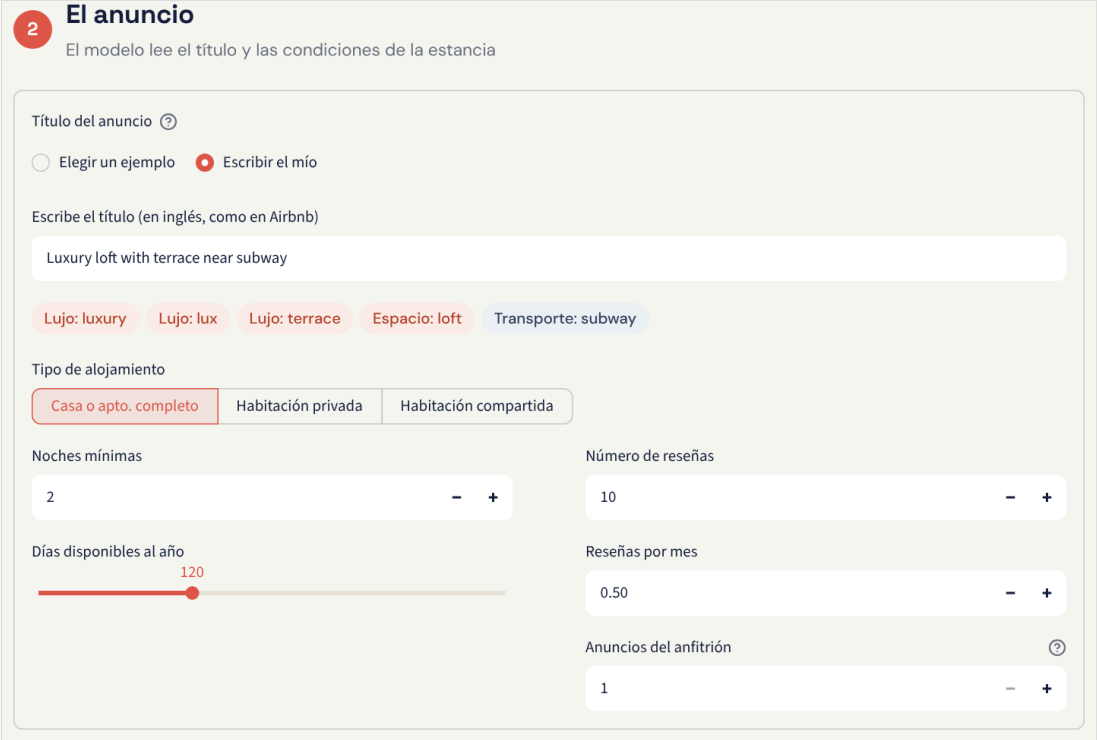


Figura 6c. Paso 2: título del anuncio con las palabras clave detectadas y datos de la estancia

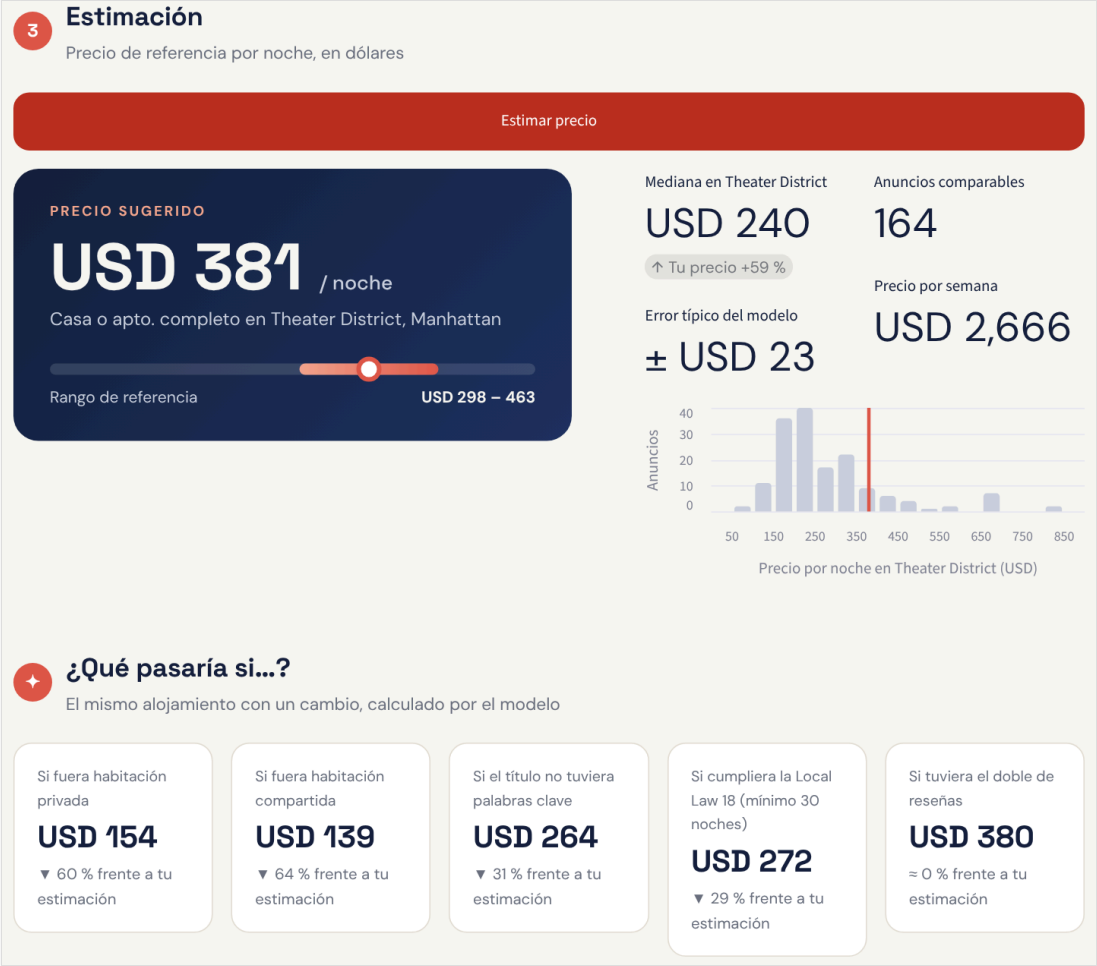


Figura 6d. Paso 3: precio sugerido con su rango, comparación con el barrio y escenarios «¿Qué pasaría si...?» calculados con el modelo

5.2 Monitoreo: definir tiempo de monitoreo

El modelo se entrenó con datos de 2019, y los precios cambian con el tiempo (inflación, temporada, regulación del alquiler de corta estancia, como la Local Law 18 de 2023). Antes de usarlo en producción debe reentrenarse con una extracción reciente de Inside Airbnb. Una vez en uso, se propone el siguiente monitoreo:

- **Frecuencia:** mensual durante los primeros 6 meses y trimestral después, comparando las predicciones con los precios reales de una nueva extracción de datos.
- **Medidas de desempeño:** MAE, MedAE y MAPE sobre los anuncios nuevos. Umbral de alerta: MAE mayor que 64 USD o MedAE mayor que 26 USD (un deterioro de alrededor del 15 % respecto a la evaluación actual).
- **Deriva de los datos:** cambios en la mediana del precio, en la proporción de tipos de alojamiento y de distritos, en la distribución del micro-barrio y en las palabras de los títulos, y aparición de valores fuera de las reglas de calidad.

5.3 Cronograma de mantenimiento

Tabla 13. Cronograma de mantenimiento

Actividad	Frecuencia	Descripción
Monitoreo del desempeño	Mensual (6 meses), luego trimestral	Calcular MAE, MedAE y MAPE con datos nuevos y compararlos con los umbrales
Revisión de deriva de datos	Mensual	Comparar distribuciones de precio, tipo de alojamiento, distrito y barrio con las del entrenamiento
Actualización de datos y reentrenamiento	Trimestral o al superar un umbral	Nueva extracción de Inside Airbnb, preparación de datos y reentrenamiento del pipeline final (`train_model.py`)
Reselección completa de modelos	Anual	Repetir validación cruzada, ANOVA/Tukey e hiperparametrización con los 7 modelos (notebook `Modelo_Airbnb_NYC`)
Mantenimiento de la aplicación	Semestral	Actualizar librerías (scikit-learn, XGBoost, Streamlit) y verificar con las pruebas automáticas que el modelo carga y predice correctamente